

OPC Code Atlas

How the repository builds a simulated recommendation world in which the logger sees users and items through distorted vectors, learns new policies from the logger's data, and grades them against the hidden truth. The two sections this revision centers on are the kinds of representation bias the study covers and the mechanisms that can correct each one.

Repo [TauRoed/OPC](#), branch [CRM](#) Code at [603cc2c](#) (pushed), 28 Sep 2026 Functions are named, not line-numbered Paths are relative to the repo root
PDF export of the [published page](#), 29 Sep 2026

CONTENTS

[The question](#) [Pipeline](#) [Representation bias](#) [Correction mechanisms](#) [Bias × correction](#) [The four arms](#) [Step by step](#) [Key data objects](#) [Entry points and flags](#) [Outputs](#)
[Modules](#) [Tests](#) [Open risks](#) [Findings so far](#) [Plan](#)

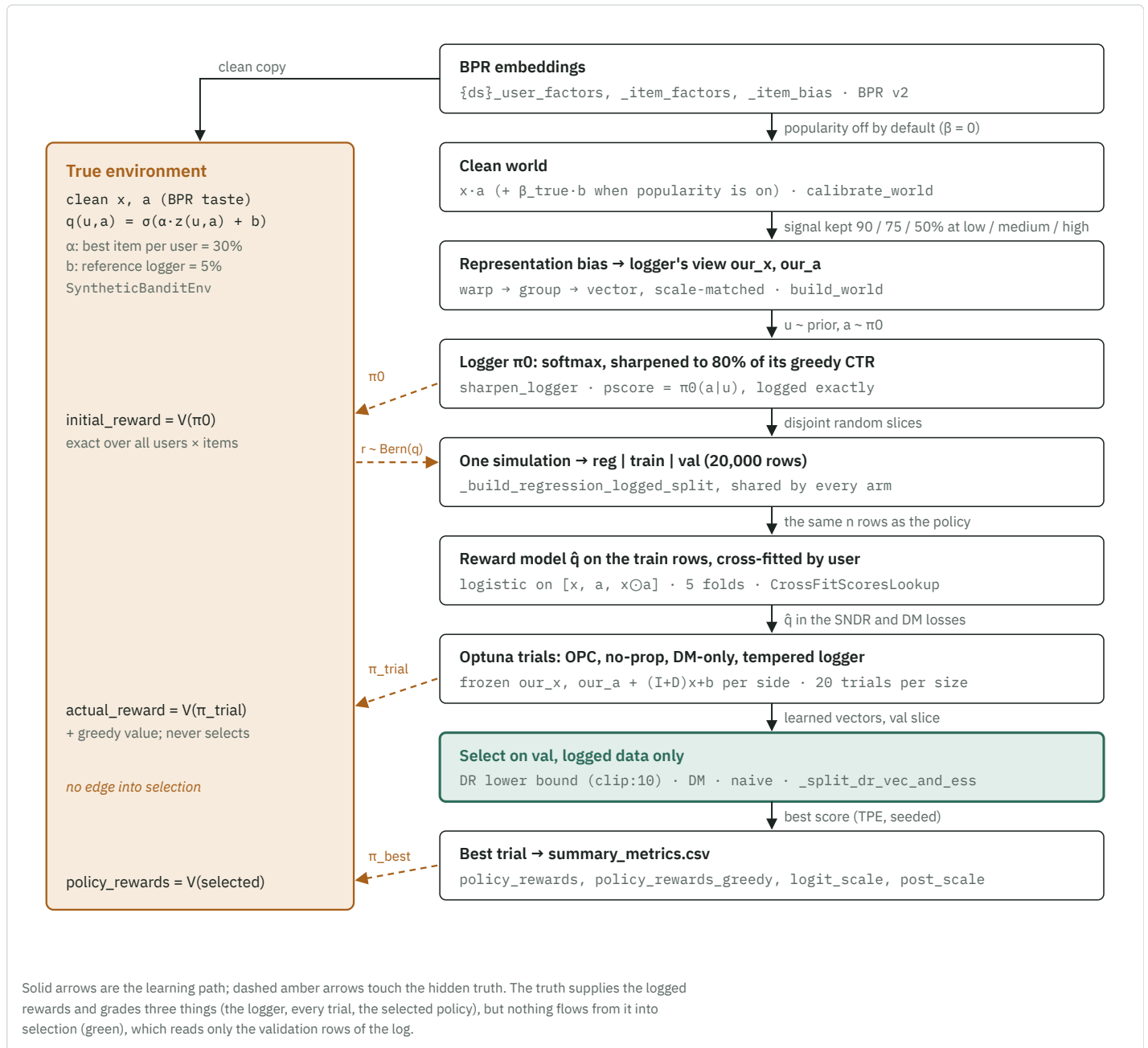
THE RESEARCH QUESTION

Can off-policy learning undo a logger's representation bias?

A deployed recommender scores items with learned user and item vectors. Those vectors are wrong in structured ways: a global misalignment of taste axes, whole user segments misread, individual tastes never learned. The logs it collects inherit those errors. The study asks which of these distortions a policy learned from the logs can correct, with which mechanism, how much data it takes, and whether logged propensities add anything beyond a good reward model. The simulator makes every distortion a dial and computes every policy's true click rate exactly.

The pipeline at a glance

A study is a grid of *conditions* (dataset × bias configuration × seed, run at several training sizes). Each condition runs the chain below for every arm. The design rests on the wall between the two columns: the true environment produces the rewards and grades the results, and nothing that chooses a policy ever reads it.

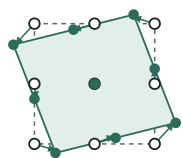


Representation bias: the distortions we cover

The truth is BPR's clean user and item vectors. The logger, the reward model and every learned policy see biased copies instead. Three kinds of bias are mixed in, in this order, on both sides (users and items), each rescaled to the clean vectors' size so a level means a share of signal replaced and never an inflation of vector length:

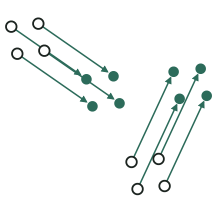
`biased = (1 - ε) · current + ε · target`, applied for warp, then group, then vector, and rescaled to the clean RMS after each step (`utils/representation_bias.py · SideBias.biased`).

Warp
one linear map for the whole side



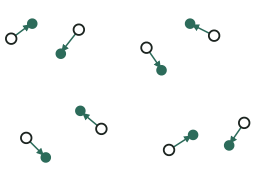
Taste axes misaligned for everyone:
a stale or globally mis-trained model
`target = x · W, W ~ N(0,1)^(d×d)`

Group
one offset per group of users or items



segment A segment B
Whole segments' tastes misread
`target = template[group(x)], k-means`

Vector
independent noise per user and item



Individual tastes never learned:
sparse users, cold items
`target = N(0, I) per vector`

Open circles are clean vectors, filled green dots the logger's biased view. A warp moves everyone by one shared map, so its structure is global. A group offset moves a whole segment together. Vector noise moves each user or item on its own, so no shared structure is left to learn from.

Levels, configurations and calibration

The strength ϵ of each type is calibrated per dataset so that **all three together** keep 90%, 75% or 50% of the taste signal at low, medium and high bias. Signal kept is the mean, over users, of the correlation across items between biased and clean taste scores. A single type at level L alone keeps κ_L of the signal (about 0.97, 0.94 and 0.88). A configuration names the three levels, for example `high/none/none` for warp only.

CONFIGURATION	WHAT IT ISOLATES	LOGGER'S GREEDY RANKING REACHES (SHARE OF THE 30% CEILING, ML / KUAIRAND / ANIME)	COVERED IN
<code>none</code>	control: no representation bias, only the logger's exploration	100%	dev grid, final report
<code>low · medium · high</code>	all three types together, 90 / 75 / 50% signal kept	80–89% · 59–73% · 38–47%	Test 2 (medium, high); final report (all)
<code>high/none/none</code>	warp only	73% / 82% / 73%	dev grid, final report
<code>none/high/none</code>	group only	77% / 87% / 79%	dev grid, final report
<code>none/none/high</code>	vector only	72% / 80% / —	dev grid, final report
<code>--pop-strength, --logger-pop-strength</code>	popularity: the truth weighs BPR's item bias b by β_{true} , the logger by β_{log} (over-exposure when $\beta_{\text{log}} > \beta_{\text{true}}$)	—	PLANNED RUNS (the dial exists)

The logger that collects the data

A softmax over the biased scores, sharpened per condition until it earns **80% of its own greedy CTR** (`--logger-greedy-share 0.8, sharpen_logger`). At that setting its temperature is about $T/4.2$ and it spreads over 110–260 items per user. It mostly exploits its ranking, which the bias has made wrong, and explores near the top of it. The spread temperature T , at which the clean logger covers half the catalog, only calibrates the click model. `off` restores the older near-uniform logger, which earned about 26% of its greedy value.

Correction mechanisms

A learned policy can fight the bias at three points: in what the policy can represent, in the objective it is trained on, and in how one of many trained candidates is chosen. Chips mark what exists today and what is planned.

1 · The policy class

What distortion of the vectors can the learned policy undo?

Linear correction

DEFAULT

($I + D$)- $x + b$ on the user side and on the item side, starting exactly at the logger ($D = 0$, $b = 0$). It can invert a warp exactly and remove a shared shift.

```
--policy-transform linear · GlobalLinearCorrection
```

Linear + MLP

OPTION

The linear correction plus a zero-initialized MLP residual on the raw vector, with no LayerNorm and no dropout. Can bend the space nonlinearly, for example around group boundaries.

```
--policy-transform linear+mlp
```

MLP (older)

LEGACY

$x + \text{MLP}(\text{LayerNorm}(x))$, random start. Kept to reproduce earlier runs.

```
--policy-transform mlp
```

Sharpness: logit scale, post-hoc tempering

OPTIONS

A learnable s in $\text{softmax}(s \cdot u \cdot a / T)$, or s chosen after training on validation from 0.25–16. These change how concentrated the policy is, never its ranking.

```
--learn-logit-scale · --post-temper
```

Learnable popularity weight

AUTO

With a popularity column, the policy learns its own weight on b , starting at the logger's. This corrects over- or under-exposure.

```
CFModel(pop_weight)
```

Per-group and per-item offsets

PLANNED

A learned offset per user group (k -means on the biased vectors) and per item, pulled toward zero with strength shrinking as the group's or item's logged data grow (partial pooling). Aimed at group bias and at the part of vector bias that has enough data behind it.

2 · The training signal

How is a biased log turned into an objective?

Propensity correction (OPC)

DEFAULT ARM

The reward model's value of the policy plus an importance-weighted correction on the logged rows. This removes the logger's exposure bias where the logger explored. The working development default (since f5cade9, 27 Sep) is DR, differentiated directly, with harmonic:0.1 weights, and selection stays clip:10. It is the working method, not the final paper choice. Legacy minibatch SNDR, global SNDR (a stop-gradient surrogate, not exact SNDR) and the log trick remain for reproducibility. In paired development runs, DR beat both SNDR variants on nearly every identical configuration.

```
--policy-losses dr (default) | sndr · --sn-scope batch|global · --opc-gradient direct (default) | log-trick
```

Importance-weight transform

OPTIONS

The options are none (raw DR), clip: M , shrink: λ (Su et al. 2020) and harmonic: λ (Metelli et al. 2021): $w/(1 - \lambda + \lambda w)$, at most $1/\lambda$, and trained by its direct gradient. Under the log trick, a transform g optimizes $H(w) = \int g(t)/t \, dt$ rather than the named estimate; for shrink:100 that is $10 \cdot \arctan(w/10)$. In development runs, smooth corrections beat raw DR from 25k up. The working default is harmonic:0.1; shrink:100 is the prespecified standard comparison, and none (raw DR) the reference.

```
--train-weights (default harmonic:0.1) · docs/training_losses.md §3.4, §9
```

Reward model alone (DM-only)

BASELINE

Maximizes the policy's value under $\hat{q}$. It reaches items the logger never showed if $\hat{q}$ extrapolates well; no propensities.

```
dm loss · --methods dm
```

Logged clicks as they are (no-propensity)

BASELINE

Raises the probability of clicked logged items. It inherits the logger's exposure (it favors $\pi_0 \times q$) and stays near the logger.

```
naive loss
```

The reward model $\hat{q}$

DEFAULT

Logistic regression on $[x, a, x \odot a]$ of the biased vectors, so rankings can differ between users. It is fit on the same n training rows as the policy and cross-fitted by user (5 folds).

```
--reward-features interaction · --reward-data train · --crossfit-folds 5
```

Bilinear reward model $x^T W a$

PLANNED

A full (ridge or low-rank) bilinear form represents a warp exactly; $x \odot a$ captures only its diagonal.

Cluster-level weights

PLANNED

Importance weights over item clusters (MIPS, OffCEM) or a two-stage policy (POTEC), for catalogs where item-level weights collapse (msd, lastfm).

3 · Selection

Which of 20 trained candidates is deployed?

DR lower confidence bound

OPC

Mean $-t \cdot SE$ of the doubly robust estimate on 20,000 validation rows, weights clipped at 10. It captures $\geq 99.7\%$ of the best trial's gain.

--select-weights clip:10 · --optuna-selection ci_low

Reward-model score

DM-ONLY

The DR score with every weight 0.

select_estimator="dm"

Naive score

NO-PROPENSITY

Mean of $r \cdot \pi$ on validation.

Tempered logger

BASELINE

No training at all: the logger's logits $\times s$, with s chosen by the DR score. It measures how much of any gain is sharpening alone.

--methods tempered_logger

Every rule, after the fact

LOGGED

Every trial of every arm carries 14 selection scores (13 weight rules and DM), so any selection rule can be replayed on any training rule without new runs.

--log-select-weights

FOCUS 3

Which mechanism can fix which bias

Each cell rates a correction mechanism against a bias type, by construction and by the oracle bound. The oracle trains the policy class directly on the true click probabilities, with unlimited data (ml, high level of each type alone).

BIAS TYPE	LINEAR CORRECTION	GROUP / ITEM OFFSETS	REWARD MODEL $X \odot A$	BILINEAR REWARD MODEL	PROPENSITY CORRECTION	ORACLE BOUND, ML (GREEDY LOGGER $\rightarrow$ LINEAR $\rightarrow$ + GROUP OFFSETS)
Warp	exact	not needed	diagonal only	exact · planned	where π_0 explored	22.1 $\rightarrow$ 29.7 $\rightarrow$ 29.2 (ceiling 30.4)
Group	shared shift only	targets it · planned	partial	partial	where π_0 explored	23.5 $\rightarrow$ 28.2 $\rightarrow$ 28.9
Vector	de-noising only	items with data · planned	no structure	no structure	where π_0 explored	21.8 $\rightarrow$ 24.9 $\rightarrow$ 26.3
Popularity exposure	—	—	sees b	sees b	removes exposure	planned runs; policy learns its own weight on b
Logger sharpness (not a bias)	logit scale or post-hoc tempering; the tempered-logger baseline measures this part				noisy for flat loggers	sharpening alone: logger 15.5 $\rightarrow$ 19.3 (medium)

Warp is almost fully fixable by the linear class (7.6 of 8.3 lost points back), group bias mostly, vector noise only partly. Propensities help wherever the logger's exploration reached, so a sharper logger helps less at small data (see Findings). The dev grid measures these cells with actual learning: linear vs linear+MLP vs MLP, with and without post-hoc tempering, for each type alone.

The four arms

All arms get the same splits, the same reward model, the same search budget (20 trials per size), the same seed and, where they apply, the same weight rules. What differs is how each turns the log into an objective and a selection score.

ARM	TRAINING OBJECTIVE	SELECTION SCORE	WHAT IT ISOLATES
OPC DEFAULT	DM($\hat{q}$) + importance-weighted correction: DR, direct gradient and harmonic:0.1 weights (working default); shrink:100 is the standard comparison; legacy and global SNDR and the log trick are kept for reproducibility	DR lower bound, clip:10	the method: reward model + propensities
No-propensity DEFAULT	naive: $-\text{mean } r \cdot \pi(a x)$	naive mean $r \cdot \pi$	learning from clicks without correcting exposure
DM-only OPT-IN	$-\sum \pi(a x) \cdot \hat{q}(x,a)$	$\hat{q}$ (all weights 0)	what a good reward model does alone
Tempered logger OPT-IN	none: logger's logits $\times s$, s in 0.5–64	DR lower bound	sharpening without any correction

Step by step through one condition

Order of execution inside `training/run_full_study.py` · `_run_condition` and `training/trainer_trials.py` · `regression_trainer_trial`. The parallel and H1 runners call the same function.

#	WHAT HAPPENS	WHERE
1	Seed and pin. Deterministic kernels, fixed CPU threads, <code>seed_everything(seed)</code> ; every later stream uses <code>derive_seed(seed, labels...)</code> .	<code>utils/seeding.py</code>
2	Build the world. Calibrate once per dataset and seed (bias strengths ϵ , spread temperature T , click model α and b), then for this configuration mix the bias into the logger's view and sharpen the logger.	<code>calibrate_world</code> · <code>build_world</code> · <code>sharpen_logger</code>
3	Simulate and split. One log per train size (reg + train + 20,000 validation rows), cut into disjoint slices, cached and shared by every arm.	<code>LazyRegressionSplitCache</code>
4	Reward model. Fit on each size's training rows. In the default mode it is cross-fitted by user in 5 folds: each fold's model sees the other folds' rows. The external 50k-row slice is still drawn, so splits match the older mode.	<code>fit_shared_regression_bundle</code> · <code>_crossfit_bundles</code>
5	Initial reward. Exact true value of the logger, and of its greedy ranking.	<code>_dataset_log_constants</code> · <code>calc_greedy_reward</code>
6	Optuna trials per arm. Each trial reseeds, samples lr , epochs, batch size and decay, and builds a <code>CFModel</code> on the frozen biased vectors with the chosen correction. It then trains with the arm's loss, where the training loss reads the cross-fitted $\hat{q}$.	<code>regression_trainer_trial</code> · <code>CFModel</code> · <code>train</code>
7	Optional post-hoc tempering of the trained policy (logits $\times s$ chosen on validation).	<code>_post_temper</code>
8	Score on validation (logged data only) with the arm's selection score; Optuna maximizes the lower bound. The trial also logs its true value, its greedy value, the weight diagnostics and the 14 re-selection scores. None of these is used to select.	<code>_split_dr_vec_and_ess</code> · <code>_selection_score_variants</code>
9	Best trial kept (no retrain). With TPE (the default), its hyperparameters are enqueued as the first trial at the next train size. With <code>--sampler random</code> there is no warm start, so every run of the grid trains the same configurations with the same seeds (paired comparisons).	<code>study.enqueue_trial</code>
10	Write outputs for every arm.	<code>_finalize_summary_df</code>

Key data objects

NAME	WHAT IT IS	USED FOR
<code>emb_x</code> , <code>emb_a</code> TRUTH	clean vectors (with a popularity column when popularity is on)	the click model; the oracle reward model
<code>our_x</code> , <code>our_a</code>	the biased view	the logger, the policy's frozen starting vectors, the reward model's features
<code>policy_temperature</code>	the logger's sharpened temperature T/f	the logger and every learned policy (which start at the logger)
<code>pscore</code>	$\pi_0(a u)$ of the logged action, exact	importance weights
<code>q_x_a</code> TRUTH	dense true $q(u,a)$ when $\leq 25M$ cells	exact values, reward draws
<code>RegressionScoresLookup</code> , <code>CrossFitScoresLookup</code>	$\hat{q}$ rows for batches: dense or closed form; cross-fitted per user	DM terms in training and selection
<code>world</code> record	every calibrated number: ϵ per type and level, signal kept, α , b , T , the logger's sharpness, effective items, greedy CTR	<code>run_meta.json</code> , summary columns

Entry points and the main flags

COMMAND	PURPOSE
<code>python -m BPR.generate_artifacts · BPR.evaluate</code>	fit BPR v2 per dataset (factors + item bias); test-set Recall, NDCG and MPR
<code>python -m training.run_full_study(_parallel)</code>	the study; the parallel runner pools conditions across workers and GPUs
<code>python -m training.characterize_world</code>	print a dataset's calibrated world per bias configuration
<code>python -m training.oracle_repair</code>	the oracle repair bound: the learner's policy class fit to the true click model (structural recoverability)
<code>python -m training.oracle_validation</code>	checks the oracle bounds: the same class, past the old learning-rate edge, with a longer budget
<code>python -m training.analyze_recoverability</code>	recoverability tables: <code>stage1</code> (oracle) and <code>stage2</code> (learned arms as a fraction of the oracle repair)
<code>python -m training.representation_report</code>	every figure and table of the experimental report, from the committed summaries

COMMAND	PURPOSE	
<code>python -m training.run_h1_study</code>	the H1 grid (bounded-error reward model); still on the external reward model	

FLAG	DEFAULT	CONTROLS
<code>--bias-configs</code>	low medium high	levels per type, e.g. <code>high/none/none</code>
<code>--logger-greedy-share</code>	0.8	logger sharpness (<code>off</code> = spread logger)
<code>--pop-strength</code> , <code>--logger-pop-strength</code>	0, = truth	popularity in the truth and in the logger
<code>--methods</code>	opc no_propensity	add <code>dm</code> , <code>tempered_logger</code>
<code>--policy-transform</code>	linear	linear, linear+mlp, mlp
<code>--learn-logit-scale</code> , <code>--post-temper</code>	off	sharpness learned in training, or chosen after training
<code>--reward-features</code>	interaction	[x, a, x⊙a] or concat [x, a]
<code>--reward-data</code> , <code>--crossfit-folds</code>	train, 5	$\hat{q}$ on the policy's own rows, cross-fitted; <code>external</code> = separate 50k slice
<code>--policy-losses</code> , <code>--sn-scope</code>	dr, batch	OPC objective: dr (working default); snr with the batch (legacy) or global scope
<code>--opc-gradient</code>	direct	how OPC's loss is differentiated; <code>log-trick</code> reproduces the runs before f5cade9
<code>--train-weights</code> , <code>--select-weights</code>	harmonic:0.1, clip:10	weight transforms in training and in selection: none, clip:M, shrink: λ , harmonic: λ
<code>--sampler</code> , <code>--stage</code>	tpe, development	random = paired replay of identical configurations; the run's purpose, recorded with the results
<code>--val-size</code>	20000	fixed validation rows (<code>0</code> = fraction rule)

The runs before 26 Sep reproduce exactly with `--reward-features concat --train-weights none --select-weights clip:1 --policy-transform mlp --logger-greedy-share off --reward-data external --val-size 0` (checked end to end against ce827f5).

Outputs and what the columns mean

COLUMN	MEANING
<code>policy_rewards</code> TRUTH	true CTR of the selected policy; at train size 0, the logger's
<code>policy_rewards_greedy</code> , <code>actual_reward_greedy</code> TRUTH	true CTR of the policy's top item per user: rankings compared without sharpness
<code>actual_reward</code> TRUTH	each trial's true CTR (never used to select)
<code>value</code> , <code>r_hat</code>	each trial's selection score (lower bound) and point estimate
<code>sel_ci_low[spec]</code> , <code>sel_r_hat[spec]</code>	re-selection scores under 14 rules, every arm
<code>ess</code> , <code>ess_raw</code> , <code>diag_*</code>	effective sample size of the transformed and raw weights; weight profile; DM vs correction split
<code>logit_scale</code> , <code>post_scale</code>	learned and post-hoc sharpness factors
<code>logger_greedy_share</code> , <code>logger_sharpness</code> , <code>logger_greedy_ctr</code>	the logger of the condition
<code>reward_data</code> , <code>crossfit_folds</code> , <code>post_temper</code> , ...	the settings of the run, one column each

Condition folders name every non-default setting, e.g.
`dataset=m1_bias=medium_ctr=0.05_seed=100_qhat=train_cf=5_val=20000`.

Modules

utils/

- `representation_bias.py`: calibration, the three bias types, logger sharpness, world record
- `simulation_utils.py`: click model, `generate_dataset`, logging, exact and greedy values
- `importance_weights.py`: none, clip, shrink, harmonic, dm; ESS
- `policies.py`, `seeding.py`, `noise_snr.py`

models/

- `models.py`: `CFModel` (logit scale, popularity weight), corrections, `RegressionModel`
- `custom_losses.py`: snr (batch or global scope), dr, dm, ipw, naive, kl, crm; log-trick or direct
- `estimators.py`: post-hoc DM, DR, SNIPW, SNDR (clip, shrink, harmonic)

training/

- `trainer_trials.py`: splits, reward models, cross-fitting, trials, selection, tempering
- `run_full_study(_parallel).py`: conditions, arms, flags, outputs
- `oracle_repair.py`: the oracle repair bound;
`oracle_validation.py`: its check;
`analyze_recoverability.py`: recoverability, per-dataset and gap tables
- `memory_budget.py`: workers per GPU from each condition's estimated peak and the free memory
- `representation_report.py`: the experimental report's figures and source tables
- `training_utils.py`, `characterize_world.py`, `run_h1_study.py`

BPR/

- `bpr.py`, `generate_artifacts.py`: BPR v2 with item bias, numpy trainer
- `evaluate.py`: test-set metrics on CPU and GPU
- `bpr_config.py`: per-dataset recipes and the default six datasets

Tests

424 tests pass with the GPU; 401 pass and 8 are skipped with it hidden (at 603cc2c). Every change since 25 Sep was also checked bit for bit against the previous commit with the new option off. New checks on 27 Sep:

- each objective variant's gradient against literal full-data autograd;
- equal row weights across the DataLoader's short final batch;
- identical replayed configurations;
- log-trick and direct gradients for every transform;
- the harmonic transform: formula, limits, the gradient bound and serialization;
- the oracle repair: the learner's class, starting exactly at the logger, deterministic; the recoverability tables regenerate byte for byte;
- a one-size run reproduces that size's trials of a multi-size run, which is what lets slices pair with the main runs;
- the worker planner against what a study run holds: dense $\hat{q}$ copies during training and at their peak, and plans for one, two and four GPUs;
- the oracle validation reproduces the Stage 1 winners exactly and extends past a winning edge; the follow-up tables never lower a bound and decompose the target gap exactly.

The world

- calibration targets on the full population; nested levels; seeds
- logger sharpness hits its share exactly; only the logger changes
- popularity column; exact and greedy values on CPU and GPU

Learning and selection

- device scoring equals the numpy reference, CPU and GPU
- weight transforms; DM loss; logit scale; corrections start at the logger
- cross-fitting: folds never see their users, fast forms equal routing

The study

- every arm runs and logs its scores through the study path
- reward-data modes, run keys, CLI defaults, skip rules
- end-to-end reproducibility per seed

Open risks

Found by reading the code; each is a hypothesis for a targeted test. Fixed items are kept, faded, for the record.

■ may change main results ■ post-hoc or robustness ■ reporting

The working defaults are not the paper's choice yet. `custom_losses.py` · `dr_correction` · `docs/training_losses.md` §3.4, §9

Legacy SNDR's objective depends on the batch size. Global SNDR removes that, but it is a stop-gradient, epoch-stale surrogate. DR has no normalizer. Under the log trick, a weight transform optimizes $H(w)$ rather than the named estimate; `--opc-gradient direct` optimizes the estimate itself. All of these are implemented and characterized. Since f5cade9 (27 Sep) the working defaults are DR, the direct gradient and harmonic:0.1. The runs before then are reproduced with `--policy-losses sndr --opc-gradient log-trick --train-weights shrink:100`. The paper's method is frozen only at the protocol freeze.

Best hyperparameters carry over between train sizes (TPE). `study.enqueue_trial`

Results at different sizes are not independent, and the order of sizes matters. `--sampler random` removes the warm start for paired comparisons.

Some defaults differ from what the studies used. `run_full_study(parallel)` · `_run_condition` · `trainer_trials`

The CLI defaults to `--sampler tpe`, which is unpaired and warm-starts each size, so paired studies need `--sampler random`. `--learn-logit-scale` is off, the default seeds are 0/1/2 (development used 100/101), and DM-only and the tempered logger are opt-in. Direct callers of `_run_condition` get an external $\hat{q}$ with no cross-fitting unless they pass `reward_data`, and the trainer API falls back to shrink:100 weights. Pass these explicitly. The details are in `docs/roee_handoff_20260928.md`.

An oversubscribed GPU slows down silently (WSL2). `memory_budget.py` · `run_full_study_parallel`

Under WSL2, workers that ask for more GPU memory than is free do not fail. The driver spills into shared system memory: the GPU stays 100% busy at about 130 W instead of 250–290 W, every worker slows about 50×, and no error is raised. This was confirmed on 27 Sep with the Windows shared-memory counter (anime at 4 workers). The worker planner now sizes each condition from its own budget. The estimate is the training step at its largest batch + its dense $\hat{q}$ copies + 1.5 GiB. There are 3 dense copies with cross-fitting and a per-size refit, because the next size's lookup is built while the previous one is still held. Each GPU then takes $\text{floor}(0.75 \times \text{free} / \text{peak})$ workers, over any number of cards, with `--max-workers` as the cap (f2e03b5, 05222f8). anime now runs 2 workers on the 48 GB card with no spill. Still open: in GPU mode host RAM is not planned, and an anime worker holds about 14.5 GiB. On a large multi-GPU machine RAM can bind first. Results are unaffected, because runs are deterministic.

One non-finite gradient fails the whole condition. `training_utils.py` · `clip_grad_norm_error_if_nonfinite=True`

raises inside the Optuna objective, which is not caught per trial.

With a flat logger, OPC does not sharpen.

With the spread logger OPC's learned scale stays near 7, while the tempered logger chooses about 60. Post-hoc tempering (`--post-temper`) addresses this and is tested in the dev grid.

A logging uniform mix enters the initial reward but not the start point.

Only matters with `--logging-uniform-mix > 0` (off by default).

Labels.

The post-hoc `ipw` column is self-normalized; the trial column `q_error` is the validation standard error.

Fixed this week. FIXED

Clicks capped at the CTR, popularity-dominated vectors and noise that inflated vector length (simulator revamp). A reward model with one ranking for every user (interaction features). Selection clipped at 1. One reward model reused across sizes and fit on extra data (budget-fair $\hat{q}$). No-propensity trials missing re-selection scores (76a29e6). On 27 Sep: the DataLoader's final short batch was upweighted by up to 4.8× (b5efc7d); `--no-log-trick` never reached OPC (documented, 3bed8de); post-hoc estimators used raw weights for an unknown transform, which now raises (d40aaef).

Findings so far

All development runs. The representation-repair numbers are in

`docs/representation_repair_dev_20260927.md`; earlier ones in the separate results report.

Structure decides what a linear repair can fix.

We trained the learner's own linear class on the true click model. It recovers 0.99 of the ranking loss from warp, 0.68 from group and 0.50 from vector (high level), and 0.67–0.69 from the combined biases. The order warp > group > vector held in all 6 dataset × seed pairs. A validation with learning rates past the old grid edge and up to 9× the optimization budget raised these by only 0.005–0.010 on average (at most 0.025 in any world), which is not material (`docs/representation_repair_followup_20260927.md`).

What is left unrecovered: learning for warp, structure for vector.

The greedy representation loss splits three ways: a part the linear class cannot express (structural gap), a part it can express but OPC did not learn (learning gap), and the part OPC recovered. At 100k the split is warp 0.01 / 0.50 / 0.50, group 0.32 / 0.42 / 0.26, vector 0.50 / 0.31 / 0.19, and combined about 0.32 / 0.34 / 0.34. At 5k the learning gap is the largest part for every bias except vector.

The headline margins replicate across datasets.

OPC beats DM-only for warp by 0.8–1.2 points at every size. From 25k it also leads for group and vector, by 0.1–0.7, on each of ml, kuairand and anime. anime is the least structurally recoverable (vector 0.31 vs 0.58–0.61), which pulls the aggregate vector numbers down. The aggregate DM-only lead at combined high, 5k, comes from kuairand and anime.

Logged learning recovers a quarter to a half of that.

OPC reaches 0.27 / 0.14 / 0.07 of the oracle's ranking repair at 5k and 0.50 / 0.39 / 0.37 at 100k (warp / group / vector). DM-only trails OPC for every single-type bias from 25k up, by 0.3–1.1 points with intervals above 0; the lead is steadiest for warp, about +1 point. No-propensity learning recovers at most 0.13, and sharpening alone none. With no bias OPC trails the tempered logger by 0.4–1.5 points. Training with shrink:100 instead of harmonic:0.1 changes nothing that matters (+0.06 points per trial).

Logger sharpness 0.6–0.95 does not remove recoverability (25k).

The oracle bound is the same at every share. OPC's recovery is flat from 0.8 to 0.95 and lowest at the most exploratory logger (0.6), where the logger collects fewer clicks and weights are heavier. OPC's lead over DM-only holds at every share. A logger without full support was not tested.

Much of the old gain was sharpening.

With the near-uniform logger, just sharpening it beat every trained policy at no and medium bias. So the logger is now sharpened to 80% of its greedy value, and a tempered-logger baseline reports the sharpening part separately.

The correction is real.

At the 0.8 logger (budget-fair, cross-fitted), OPC beats sharpening alone by 1.5–3.4 points at medium bias and 2.5–5.3 at high bias, 6 of 6 conditions in every cell. It also beats no-propensity by 0.8–4.2 points.

Propensities add a little over a good reward model.

OPC beats DM-only at medium bias (+0.5 to +1.1) and ties it at high bias. DM-only's earlier small-sample lead came from 50k extra rows; with a budget-fair $\hat{q}$ it fell 2 points at 5k while OPC did not move.

Logger sharpness 0.8–0.95 leaves the same room.

The final policies are the same, but a sharper logger shrinks OPC's lead over DM-only at small data. The near-uniform logger collects worse data for everyone.

OPC holds up when the reward model is weak or wrong.

OPC is much less sensitive than DM-only to shrinking the reward model's training sample, under these conditions. Going from a 50k-row $\hat{q}$ to the budget-fair one, OPC moved by at most ± 0.2 points, while DM-only lost 1.4–1.5 at 5k. With a misspecified $\hat{q}$ (one ranking for every user), DM-only fell 1.7–9.1 points and OPC's change stayed within noise. With 50k rows for $\hat{q}$, DM beat OPC at 5k.

The objective and weighting matter at the margin (development, paired).

On identical configurations:

- DR beat legacy and global SNDR, by 0.02–0.32 points per trial.
- The direct gradient did at least as well as the log trick.
- Smooth corrections beat raw DR from 25k up.
- Harmonic at $\lambda = 0.1$ –0.2 was ahead of Su shrinkage by 0.1–0.5 points per trial.

Those choices move OPC by up to about 0.6 points, comparable to the OPC – DM margins at medium bias. No method is promoted yet.

Selection works; training is the bottleneck.

Tight selection weights pick $\geq 99.7\%$ of the best trial's gain, and re-selecting any arm's trials by another rule moves its value by under 0.2 points. Cross-fitting $\hat{q}$ changed nothing measurable (the logistic model has about 100 parameters).

Plan

Status, 27 Sep	CLOSED The objective and weighting development work is done and recorded in <code>docs/decision_record_opc_objective_weighting.md</code>. The working defaults are DR, the direct gradient and harmonic:0.1, with selection at clip:10 (since f5cade9); the final paper choice is still open. The representation-repair development stages are also done and recorded in <code>docs/representation_repair_dev_20260927.md</code>: the oracle bound, learned recovery and logging support. A follow-up checked the bounds, split the results by dataset and decomposed the target gap (<code>docs/representation_repair_followup_20260927.md</code>). A code handoff since Roe's last commit (<code>docs/roee_handoff_20260928.md</code>) and one experimental report (<code>docs/representation_repair_experimental_report_20260928.md</code>, figures in <code>artifacts/full_study/report_20260928/</code>) summarize everything. Comparisons with prior work (CausE, BLOB) are next, pending priorities. Audits of those methods are external, standalone work; nothing here integrates or depends on them. No dev or final grid has been launched. All runs so far are development runs; confirmatory runs will use fresh seeds.
Dev grid	NEXT linear vs linear+MLP vs MLP corrections, with and without post-hoc tempering, for no bias, medium, high, and warp, group or vector alone (ml, kuairand, anime).
Final report	THEN all six datasets × no / low / medium / high bias × warp-only, group-only, vector-only × 5k, 25k, 100k, four arms, fresh seeds.
Model changes	PENDING bilinear reward model; per-group and per-item offsets with partial pooling; popularity runs; cluster-level weights for large catalogs; Switch-DR; validation as a share of the budget; aligning H1 with the new defaults; real-data checks (Open Bandit Dataset, KuaiRec's full matrix, KuaiRand's random exposures, Coat, Yahoo! R3).
Assumption	Logged propensities are known exactly, as in most OPE and OPL work; estimating them is outside this study.
Deferred	Comparing policies by their exploitation (greedy) part alone. The greedy value is logged on every trial for when we return to it.